

АНАЛІТИЧНИЙ ЗВІТ

Комплексний аудит та оцінка ризиків плану розвитку архітектури Brain_KnowledgeVault

Об'єкт аналізу:	Проект sSOFTINn/Brain_KnowledgeVault (Файл архітектурного плану: plan.md)
Ціль аудиту:	Логічна валідація, виявлення прихованих технічних розривів та безпекових зауважень
Концептуальний стек:	Local-first RAG, Obsidian, Wiki LLM (Паттерн Карпати), Graphify
Дата оцінки:	29 червня 2026 року
Статус плану:	Рекомендовано до коригування (виявлено критичні архітектурні розриви)

1. ВСТУП ТА ЗАГАЛЬНА КОНЦЕПТУАЛЬНА ОЦІНКА

Наданий план розвитку системи Brain_KnowledgeVault демонструє глибокий інженерний підхід до побудови персональної local-first екосистеми управління знаннями. Базовий фундамент (Фази 1–5), що спирається на використання інструментів перевірки цілісності (vaultctl doctor), детермінованих міграцій та сховища на базі SQLite з повнотекстовим пошуком FTS5, є повністю зрілим та архітектурно обґрунтованим.

Ключовий керівний принцип проекту — «спочатку безпека, відтворюваність і джерела; тільки потім LLM» — є критично правильним. Він запобігає класичним помилкам розробки сучасних RAG-систем, коли інтелектуальні агенти підключаються до неструктурованих, зашумлених або невалідованих даних, що призводить до неконтрольованих галюцинацій та витоку інформації.

Проте, детальний перехресний аналіз подальших етапів (Фази 6–13) виявив кілька суттєвих логічних розривів, архітектурних протиріч та прихованих безпекових ризиків, які потребують усунення до початку активної фази кодингу.

2. КРИТИЧНІ ЛОГІЧНІ ТА АРХІТЕКТУРНІ РОЗРИВИ

2.1. Хронологічний розрив: Graph Layer (Phase 11) проти LLM Answer (Phase 9)

У поточній послідовності плану побудова графового шару та інтеграція з принципами децентралізованого мапінгу зв'язків віднесена на **Фазу 11**, у той час як формування відповідей моделлю (Phase 9) та автоматичне створення Wiki-чернеток (Phase 10) відбуваються раніше.

Суть розриву: Головна перевага поєднання Obsidian та концепції Graph RAG / Graphify полягає в тому, що LLM під час формування відповіді або генерації нових нотаток має спиратися не на ізольовані плоскі текстові шматки (chunks), а на **топологічну карту взаємозв'язків між сутностями**. Якщо графовий індекс з'являється лише на Фазі 11, то інструменти Фаз 9 та 10 будуть позбавлені топологічного контексту, працюючи як звичайний класичний RAG. Модель буде «сліпою» до нативних `[[wikilinks]]`, створених користувачем, що нівелює головну ідею всього стеку.

2.2. Загроза каскадного витоку даних за відсутності Purge-механізму

Документ містить суворі правила безпеки для III-контексту (не індексувати папку `Private/`, ігнорувати файли з тегом `visibility: restricted`). Проте в описі інкрементального оновлення баз даних (Phase 7 та Phase 8) повністю відсутня логіка очищення індексу (Garbage Collection) у разі зміни статусів доступу.

Суть розриву: Розглянемо сценарій: користувач створює публічну нотатку, система розбиває її на чанки і заносить у `rag.sqlite3`. Через деякий час користувач переносить файл у директорію `Private/` або додає у `frontmatter` мітку `restricted`. Під час наступного сканування інкрементальний індексатор просто пропустить цей файл як заборонений до читання, але його застарілі чанки **назавжди залишаться всередині бази даних RAG**. Як наслідок, локальна LLM продовжуватиме читати та використовувати секретні дані у своїх відповідях. Системі необхідний механізм каскадного видалення застарілих індексів.

2.3. Проблема «контекстуального сирітства» чанків при Heading-декомпозиції

У Фазі 7 закладено правило: «*chunk за кожним heading*». Це стандартний підхід для Markdown, але він несе прихований ризик руйнування контексту верхнього рівня.

Суть розриву: Шматок тексту, розташований під підзаголовком `### Ризики міграції` всередині довгої технічної нотатки про PostgreSQL, після відокремлення стає абстрактним набором загроз для моделі ембеддінгів. Без явного збереження всього ланцюжка батьківських тем цей чанк втрачає семантичну цінність.

3. ОЦІНКА ТЕХНІЧНИХ ТА ОПЕРАЦІЙНИХ РИЗИКІВ

ФАЗА ПЛАНУ	СУТЬ ВИЯВЛЕНОГО РИЗИКУ	КРИТИЧНІСТЬ	РЕКОМЕНДОВАНИЙ ЗАХІД БЕЗПЕКИ
Phase 7 / 8	Зависання конфіденційних чанків: Збереження текстових блоків у БД після зміни статусу файлу на Restricted або Private.	КРИТИЧНИЙ	Впровадити обов'язковий етап Purge / Garbage Collection перед кожним скануванням для очищення застарілих або заборонених шляхів.
Phase 9 / 10	Втрата топологічного контексту: Робота LLM без урахування графа зв'язків та структури МОС (Map of Content).	ВИСОКИЙ	Перенести запуск базового графового експорту (Phase 11) вище за хронологією — перед підключенням модуля відповідей LLM.
Phase 7	Контекстуальне сирітство: Втрата зв'язку чанка з назвою файлу та батьківськими заголовками.	ВИСОКИЙ	Під час індексації примусово склеювати значення поля heading_path із тілом тексту чанка (напр., [Шлях: Бази Даних -> SQLite] Текст...).
Phase 8	Ресурсне голодування CPU: Команда повного перерахунку векторів --rebuild на базах у 10k+ файлів може заблокувати систему на години.	НИЗЬКИЙ	Заборонити запуск повного перезапису ембедінгів без явного підтвердження користувача. Обробляти дані суворо по дельті через content_hash.
Phase 13	Обмеження Windows MAX_PATH: Довгі назви файлів у глибоких підпапках Obsidian викличуть збої Python-скриптів при копіюванні у Staging/.	ВИСОКИЙ	У коді утиліти vaultctl використовувати префікси довгих шляхів \\?\ для Win32 API або нормалізувати робочі шляхи через UID.

4. РЕКОМЕНДОВАНА ОПТИМІЗОВАНА ЧЕРГОВІСТЬ РЕАЛІЗАЦІЇ

Для усунення архітектурних розривів та забезпечення безшовної інтеграції Graph RAG пропонується оновити стратегічний порядок впровадження фаз:

1. Phase 6 — Backup & Restore експериментами)	(Захист та відкат системи перед
2. Phase 7 — RAG Sources Layer застарілих чанків)	(Чанкінг, SQLite FTS5 + механізм Purge

3. Phase 11 – Graph Layer (Зсунуто вгору! )	(Визначення топології знань та зв'язків між файлами)
4. Phase 8 – Embeddings & Hybrid Search	(Семантичний пошук поверх вже готового графа та ключів)
5. Phase 9 – LLM Answer Layer	(Відповіді ШІ з контекстом з тексту + суміжних вузлів графа)
6. Phase 10 – Wiki LLM Draft Tools	(ШІ як асистент для генерації нових зв'язків та МОС)
7. Phase 12 – Dashboard / UI	(Візуалізація індексів, бакапів та стану зв'язків)
8. Phase 13 – Hardening & Performance	(Стрес-тести, Windows paths, Unicode)

5. АНАЛІЗ ВІДПОВІДНОСТІ ПРАВИЛАМ CODEX (CODEX RULES)

Розділ інструкцій та обмежень для штучного інтелекту («Правила для Codex») сформульований бездоганно і є однією з найсильніших сторін плану. Слід виділити три фундаментальні рішення:

- 1. Заборона прямого редагування (Правило 6 та 7):** Вимога проводити всі генерації нотаток через буферну зону *Staging/* з обов'язковим переглядом diff-файлів захищає особистий архів користувача від випадкового псування або спотворення фактів моделлю. Система зберігає критично важливий статус *human-in-the-loop*.
- 2. Гібридна стійкість (Правило 9):** Обов'язковий архітектурний відкат до локального повнотекстового пошуку SQLite FTS5 у разі збою або відсутності векторних ембедінгів гарантує працездатність системи в будь-яких умовах автономності.
- 3. Суворе обмеження видимості (Правило 3 та 4):** Повна ізоляція конфіденційних зон на рівні ядра індексатора унеможливорює випадкове потрапляння приватних даних у контекст prompt-токенів.

6. ВИСНОВОК

Проект *Brain_KnowledgeVault* має чудову інженерну основу та чітке розуміння концепту *local-first* систем. Описані в плані кроки не потребують радикального перегляду архітектури. Для досягнення максимальної ефективності та безпеки розробнику достатньо скоригувати черговість підключення графового індексатора (перенести Phase 11 вище) та інтегрувати автоматичне видалення (*purge*) застарілих чанків з бази даних RAG під час інкрементального сканування.